ABES Engineering College, Ghaziabad

Department of Computer Science and Engineering

Session: 2026-27, Semester: 3, Sections: CSE 21 and 26

Course Code: 25CS301 **Course Name:** Object Oriented Programming with C++

Assignment 1

Date of Assignment: 13-08-2026 **Date of Submission:** 28-08-2026

General Instructions

- Attempt any 10 questions unless a different subset is announced by the instructor.
- Submit one clearly named ISO C++17 source file per question, with a brief design note and at least three labelled test runs.
- Use only Unit 1 mechanisms; do not use constructors, inheritance, polymorphism, templates, exceptions, or dynamic memory.
- Validate inputs, use meaningful identifiers, and state any additional assumptions explicitly.

S.No.	Question	KL, CO
1	**From a Monolithic Script to an Object-Based Electricity Billing System** ***Primary concepts:*** *Evolution of programming paradigms; structured vs object-oriented development; functions; classes and objects* **Scenario:** A small electricity cooperative began with one long program that reads a consumer number, units consumed, computes the bill, and prints a receipt. As the cooperative added multiple counters, repeated code and global variables made the program difficult to test. The technical team now wants a structured version and an object-based version that produce identical bills. **Programming task:** Develop one menu-driven C++ program that demonstrates the evolution from monolithic thinking to structured and object-oriented development. You do not need to reproduce a full monolithic implementation; instead, summarize its likely problems and implement the same billing rule twice: once with ordinary functions and once with a ConsumerBill class and objects. **Functional and design requirements:** • Accept consumer number, consumer name, units consumed, and tariff category for at least three sample consumers. • In the structured module, keep data in ordinary variables and use separate functions for input, calculation, and receipt printing. • In the object-oriented module, define a ConsumerBill class whose data members represent state and whose member functions represent the same three operations. • Use one common slab rule in both modules: first 100 units at Rs. 4.00, next 200 at Rs. 6.00, and remaining units at Rs. 8.00; add a fixed charge of Rs. 75. • Ensure that the structured and object-based modules produce the same amount for the same input. Do not use inheritance, constructors, or other later-unit features. • After the code, write a 150-200 word comparison explaining how control flow, data ownership, reuse, and maintainability change across the three paradigm stages. **Minimum validation evidence:** • Test boundary values of 100, 300, and 301 units. • Show one run through each menu option with the same consumer data and compare the amounts. • Reject negative units and an empty consumer name. **Submission for this question:** design note, complete .cpp source, labelled test evidence, and concept reflection.	**K4, CO1**
2	**Re-engineering a Campus Lost-and-Found Register** ***Primary concepts:*** *Structured vs OOP; classes and objects; program decomposition; state and behaviour* **Scenario:** The campus security office stores lost-item records in parallel arrays and uses unrelated functions to add, search, and display items. When one array is updated but another is not, item IDs and descriptions become misaligned. The office wants a small console prototype that shows why grouping related data and operations is useful. **Programming task:** Create two implementations of a register for up to five items. The first must follow a structured approach using parallel arrays and functions. The second must model each record as a LostItem object. Both versions must support the same operations so that their designs can be compared fairly. **Functional and design requirements:** • Store item ID, description, location found, and claimed status. • Provide operations to add a record, search by item ID, mark an item as claimed, and display all unclaimed items. • In the structured version, pass all required arrays and the current record count to the functions; avoid global arrays.	**K4, CO1**

S.No.	Question	KL, CO
	• In the object-oriented version, create a LostItem class with suitable data members and member functions, then create multiple objects. • Keep the user interface and sample data identical in both versions so that only the program organization changes. • Add a short design note that identifies the state and behaviour of an item and explains which version better prevents mismatched data. **Minimum validation evidence:** • Search for both an existing and a non-existing item ID. • Attempt to claim an item twice and display a meaningful message. • Demonstrate that a change to one object does not change the state of another object. **Submission for this question:** design note, complete .cpp source, labelled test evidence, and concept reflection.	
3	**Constructing a Well-Structured City Emergency Bulletin Program** ***Primary concepts:*** *C++ program structure; headers; declarations and definitions; functions; scope resolution operator* **Scenario:** A city control room needs a small program that records an alert message and severity level, then prints a standard bulletin. A previous trainee placed every statement inside main(), mixed declarations with output logic, and used a global name that was accidentally hidden by a local variable. **Programming task:** Build a complete C++ source file whose organization is easy for a new team member to audit. The program must visibly separate include directives, constants, function prototypes, a class declaration, main(), and the out-of-class member-function definitions. **Functional and design requirements:** • Use appropriate standard headers and qualify standard-library names consistently. • Declare a global integer named alertLevel with a safe default and create a local variable of the same name inside one reporting function. • Use ::alertLevel only where the program intentionally needs the global value; print both values with clear labels. • Define a Bulletin class with message and zone as data members and declare input() and display() inside the class. • Define both member functions outside the class using Bulletin:: and keep main() limited to coordination and function calls. • Add concise comments naming each structural region and explain the translation path from source code to executable in 5-7 lines. **Minimum validation evidence:** • Use a multi-word message and zone name to test input handling. • Show that changing the local alertLevel does not change ::alertLevel. • Compile with warnings enabled and submit a warning-free build transcript or screenshot. **Submission for this question:** design note, complete .cpp source, labelled test evidence, and concept reflection.	**K3, CO1**
4	**Lab Instrument Inspection Using Classes and Independent Objects** ***Primary concepts:*** *Classes and objects; data members; member functions; object state and identity* **Scenario:** An engineering laboratory inspects instruments before practical examinations. Every instrument has an asset ID, instrument name, last-calibration age in days, and current error percentage. The lab assistant needs a simple program that keeps each instrument's details separate and reports whether recalibration is required. **Programming task:** Design a LabInstrument class and use it to represent at least three different instruments. The program should make the difference between a class definition and individual object state visible in both code and output. **Functional and design requirements:** • Provide member functions to read details, evaluate calibration status, and display a formatted inspection report. • An instrument requires calibration when its last-calibration age exceeds 180 days or its error percentage exceeds 2.0. • Create three separate objects in main() and call the same member functions for each object. • After initial display, update only one object's error percentage and display all three reports again. • Label the object's identity using its asset ID and clearly separate identity, state, and behaviour in a short explanation. • Keep the implementation within basic classes and objects; do not use constructors, static members, inheritance, or friend functions. **Minimum validation evidence:** • Test one safe instrument, one old calibration, and one high-error instrument. • Test the exact boundary values 180 days and 2.0 percent. • Demonstrate that updating one object leaves the other two unchanged.	**K3, CO1**

S.No.	Question	KL, CO
	Submission for this question: design note, complete .cpp source, labelled test evidence, and concept reflection.	
5	**Resolving Three Meanings of baseCharge in a Canteen Billing Tool** ***Primary concepts:*** *Scope resolution operator; global, class, and local scope; out-of-class member functions* **Scenario:** A campus canteen uses a global platform charge, a class-level meal base charge stored in each plan object, and a temporary local base charge during promotional calculations. A naming collision caused the wrong amount to appear on receipts. The developer must now make every intended scope explicit. **Programming task:** Write a program that deliberately uses the identifier baseCharge in three scopes and produces an annotated receipt showing which value came from which scope. The objective is not merely to avoid the collision, but to demonstrate and explain how C++ resolves each name. **Functional and design requirements:** • Declare a global baseCharge representing the platform fee. • Define a MealPlan class containing a data member also named baseCharge and member functions calculate() and displayBreakdown(). • Declare displayBreakdown() inside the class and define it outside using MealPlan::displayBreakdown(). • Inside one member function, create a local variable named baseCharge for a promotional amount. • Print the local value, the object's member value, and ::baseCharge with unambiguous labels; compute a final bill from the intended values. • Add a scope table or short explanation describing the lifetime and lookup of each occurrence without introducing namespaces or advanced class features. **Minimum validation evidence:** • Use three distinct numeric values so an accidental substitution is obvious. • Modify the object's member value and confirm that the global value remains unchanged. • Temporarily remove one qualifier, record the changed result or compiler diagnostic, then restore the correct version. **Submission for this question:** design note, complete .cpp source, labelled test evidence, and concept reflection.	**K4, CO1**
6	**Live Match Scoreboard with Reference Aliases** ***Primary concepts:*** *Reference variables; aliasing; initialization; reference behaviour* **Scenario:** During an inter-college match, the scoreboard operator wants a short alias for the score currently being edited. A novice programmer assumes that a C++ reference can later be redirected to another team's score like a pointer. This misunderstanding creates incorrect score updates. **Programming task:** Create a demonstrator with homeScore and awayScore variables and a reference variable initially bound to homeScore. Use output and address comparisons to prove what the reference represents, then illustrate what really happens when awayScore is assigned through that reference. **Functional and design requirements:** • Initialize both scores and bind int& activeScore to homeScore at the point of declaration. • Update activeScore after valid scoring events and print homeScore after each update. • Print the addresses of activeScore and homeScore to demonstrate that the reference is an alias. • Execute activeScore = awayScore and explain why this copies a value rather than rebinding the reference. • Provide a small menu that lets the operator choose which team to update by calling functions with reference parameters instead of attempting to rebind activeScore. • Include a 100-150 word note contrasting a reference alias with an independent variable, limited to concepts used in Unit 1. **Minimum validation evidence:** • Start with unequal scores so value copying is visible. • Apply at least two updates through a reference and show the original variable changing. • Document the compiler error produced by an uninitialized non-const reference in a commented-out learning snippet. **Submission for this question:** design note, complete .cpp source, labelled test evidence, and concept reflection.	**K4, CO1**
7	**Modular Water-Supply Complaint Triage** ***Primary concepts:*** *Functions; prototypes; return values; modular decomposition; C++ program structure* **Scenario:** A municipal help desk receives complaints about no supply, low pressure, leakage, and water quality. Its first prototype repeats input and classification logic inside every menu branch. The team wants main() to act only as a controller while small functions perform clearly defined jobs. **Programming task:** Develop a menu-driven complaint triage system using ordinary functions. The design should show a clean flow from function declarations to calls and definitions, with each function having a single responsibility.	**K3, CO1**

S.No.	Question	KL, CO
	Functional and design requirements: • Create functions to read a complaint code, validate ward number, determine priority, estimate response time, and print a ticket. • Use parameters and return values meaningfully; do not communicate through global variables. • Assign Critical priority to leakage near electrical equipment, High to no supply, Medium to water quality, and Normal to low pressure. • Use a function prototype section before main() and place definitions after main(). • Keep main() focused on the menu loop and orchestration, and show at least one function calling another function. • Draw or describe a simple call hierarchy and justify each function's parameters and return type. **Minimum validation evidence:** • Exercise every complaint category and an invalid code. • Test ward numbers at the valid boundaries chosen by you. • Show that no function depends on hidden global state. **Submission for this question:** design note, complete .cpp source, labelled test evidence, and concept reflection.	
8	**Safe Discount Preview Through Pass-by-Value** ***Primary concepts:*** *Parameter passing by value; local copies; functions; return values* **Scenario:** An online bookstore lets a cashier preview several discounts before applying one. A preview must never modify the actual cart subtotal. A trainee accidentally expects changes made inside a function to persist and therefore shows the wrong final amount. **Programming task:** Implement a discount-preview module in which the subtotal and discount rate are passed by value. The function may change its local subtotal for calculation, but the caller's subtotal must remain unchanged until a returned result is explicitly assigned. **Required function contract:** `double previewDiscount(double subtotal, double discountPercent);` **Functional and design requirements:** • Read the cart subtotal and three candidate discount percentages. • Call previewDiscount() for each percentage and display the candidate payable amount without changing the original subtotal. • Inside the function, print the local subtotal before and after applying the discount; in main(), print the caller's subtotal after every call. • Let the cashier select one candidate and explicitly assign the returned value to a finalPayable variable. • Reject negative subtotals and discount values outside 0-100. • Explain, with a small memory sketch or address printout, why assignment to a by-value parameter does not modify the caller. **Minimum validation evidence:** • Use discount rates 0, 12.5, and 100 percent. • Verify that the original subtotal remains unchanged after all preview calls. • Show the final explicit assignment that commits the chosen preview result. **Submission for this question:** design note, complete .cpp source, labelled test evidence, and concept reflection.	**K3, CO1**

9	**Persistent Stock Updates Through Pass-by-Reference**
	Primary concepts: *Parameter passing by reference; reference parameters; functions; input validation*
	Scenario: A robotics laboratory issues component kits to student teams. The displayed stock must change immediately after a successful issue or replenishment. The first version passed stock by value, so every transaction appeared to work inside the function but the inventory screen never changed.
	Programming task: Write an inventory transaction program that uses reference parameters for values that must be modified in the caller. The program must distinguish between input-only parameters and output or input-output parameters.
	Required function contract:
	`void issueItems(int& stock, int requested, bool& success);`
	`void replenish(int& stock, int quantity);`
	Functional and design requirements:
	• Initialize stock in main() and repeatedly offer issue, replenish, view stock, and exit options.
	• issueItems() must reduce stock only when requested is positive and not greater than available stock; set success through a reference parameter.
	• replenish() must increase the caller's stock only for a positive quantity.
	• Print the stock before the call, inside the function, and after the call to make persistent modification visible.
	• Do not return the new stock value; the learning objective is modification through references.
	• Explain why requested should be passed by value while stock and success are passed by reference.
		K3, CO1

S.No.	Question	KL, CO
	Minimum validation evidence: • Perform a successful issue, an over-issue, a zero or negative issue, and a replenishment. • Confirm that failed transactions leave stock unchanged. • Print the addresses of the stock parameter and caller variable in one test to show aliasing. **Submission for this question:** design note, complete .cpp source, labelled test evidence, and concept reflection.	
10	**Null-Safe Sensor Calibration Through Pass-by-Address** ***Primary concepts:*** *Parameter passing by address; pointer parameters; dereferencing; null checking* **Scenario:** A field engineer calibrates temperature sensors by adding a measured offset to each reading. Some sensor channels may be unavailable, so the calibration function can receive no valid address. The program must modify valid readings in place without dereferencing a null pointer. **Programming task:** Develop a calibration utility that receives the address of a reading, validates the address, applies an offset, and reports whether calibration succeeded. Use ordinary local variables only; dynamic memory is not required. **Required function contract:** `bool calibrate(double* reading, double offset);` **Functional and design requirements:** • Create at least three sensor reading variables and pass their addresses using the address-of operator. • Inside calibrate(), check reading against nullptr before using the dereference operator. • Modify *reading only when the pointer is valid and the resulting temperature remains within a documented physical range. • Call the function once with nullptr to demonstrate defensive handling. • Print each variable's address, the pointer value received by the function, and the reading before and after calibration. • Add a concise comparison of the syntax used by value, by reference, and by address without using dynamic allocation. **Minimum validation evidence:** • Test positive, negative, and zero offsets. • Test a calibration that would exceed your accepted range and must be rejected. • Prove that the null call returns false and does not crash. **Submission for this question:** design note, complete .cpp source, labelled test evidence, and concept reflection.	**K3, CO1**

11	**One Triage Level, Three Parameter-Passing Behaviours**
	Primary concepts: *Pass by value, reference, and address; function effects; comparative reasoning*
	Scenario: A hospital training simulator uses an integer triageLevel from 1 to 5. Educators want one program that clearly shows which function calls can alter the original level and what syntax is required at the call site and inside the function.
	Programming task: Use a single triageLevel variable in main() and call three functions in a controlled sequence. Each function must attempt a visible update using a different parameter-passing technique, after which the program records the caller's value.
	Required function contract:
	`void simulateByValue(int level);`
	`void escalateByReference(int& level);`
	`bool resetByAddress(int* level);`
	Functional and design requirements:
	• Have simulateByValue() assign a different valid level and show that the caller remains unchanged.
	• Have escalateByReference() increase the original level by one without exceeding 5.
	• Have resetByAddress() set the original level to 1 after a null check and return a success flag.
	• Print a labelled trace before, during, and after every call, including address evidence where meaningful.
	• Add a comparison table covering call syntax, parameter syntax, whether null is possible, and whether caller state changes.
	• Conclude with a justified recommendation for which technique fits preview, mandatory modification, and optional-address scenarios.
	Minimum validation evidence:
	• Run once from level 3 and once from level 5.
	• Call resetByAddress(nullptr) and confirm safe failure.
	• Ensure all three functions enforce the 1-5 range.
	Submission for this question: design note, complete .cpp source, labelled test evidence, and concept reflection.
		K4, CO1

12	**High-Frequency Parking Calculations with Inline Functions** ***Primary concepts:*** *Inline functions; small reusable functions; return values; function design*	**K3, CO1**

S.No.	Question	KL, CO

	Scenario: A smart parking kiosk recalculates charge, duration, and overstay status whenever the user adds minutes. The calculations are short and frequently called, while receipt printing is comparatively long. The developer must choose sensible candidates for inline functions rather than marking every function inline.
	Programming task: Build a parking-charge program using small inline functions for compact calculations and one ordinary function for the multi-line receipt. Treat inline as a compiler request and focus on good function boundaries instead of claiming guaranteed speed.
	Required function contract:
	`inline double hoursFromMinutes(int minutes);`
	`inline double calculateCharge(int minutes, double hourlyRate);`
	`inline bool isOverstay(int minutes);`
	Functional and design requirements:
	• Calculate billable hours from minutes, rounding according to a rule you state clearly.
	• Apply an hourly rate and an overstay surcharge after 240 minutes.
	• Call each inline function repeatedly for at least five sample parking durations.
	• Implement printReceipt() as an ordinary function because it performs longer formatted output.
	• Place inline definitions where they are visible before use and keep each function limited to one small expression or decision.
	• Explain why the compiler may ignore inline and why recursive or large input/output-heavy functions are poor candidates.
	Minimum validation evidence:
	• Test 0, 59, 60, 240, and 241 minutes.
	• Verify the rounding and surcharge boundaries.
	• Submit a small table mapping each function to your inline or ordinary decision and rationale.
	Submission for this question: design note, complete .cpp source, labelled test evidence, and concept reflection.	

13	**A Unified Makerspace Estimator Through Function Overloading**
	Primary concepts: *Function overloading; compile-time selection; parameter lists; type conversion*
	Scenario: A campus makerspace estimates three different resource needs: the cost of a count of laser-cut panels, the cost of a measured cable length, and the coating needed for a rectangular surface. Staff want one meaningful operation name while retaining different input forms.
	Programming task: Create a family of overloaded estimate() functions. Each overload must model a distinct request and print which overload was selected so that compile-time resolution is observable.
	Required function contract:
	`double estimate(int panelCount);`
	`double estimate(double cableMetres);`
	`double estimate(double length, double width);`
	Functional and design requirements:
	• Use documented rates for panels and cable, and a documented coverage factor for the rectangular surface.
	• Call all overloads from a menu and display a labelled estimate with units.
	• Use identical function names but distinct parameter lists; do not attempt to overload using return type alone.
	• Include calls with integer and floating-point literals and explain which overload is selected.
	• Add one deliberately ambiguous or misleading call as a commented learning case, explain the issue, and show an explicit cast or corrected literal.
	• Reject negative measurements and counts in the appropriate overload.
	Minimum validation evidence:
	• Test zero and positive values for all three services.
	• Demonstrate that estimate(5) and estimate(5.0) select different overloads.
	• Show that changing only a return type cannot create a valid overload.
	Submission for this question: design note, complete .cpp source, labelled test evidence, and concept reflection.
		K3, CO1

14	**Courier Quotes with Optional Service Choices** ***Primary concepts:*** *Default arguments; function calls; trailing-default rule; input validation* **Scenario:** A student-run courier service normally sends a parcel by standard delivery without insurance. Express service and insurance are optional, but the most common call should remain concise. The billing function must provide sensible defaults while allowing callers to override them. **Programming task:** Implement a quote function with default values for optional service choices. Demonstrate the progressive use of arguments from the shortest valid call to the fully specified call, and explain the effect of each omission. **Required function contract:** `double quote(double weightKg, int distanceKm, bool express = false, double insuranceValue = 0.0);`	**K3, CO1**

S.No.	Question	KL, CO
	Functional and design requirements: • Define and document a base-per-kilogram charge, a distance charge, an express surcharge, and an insurance percentage. • Make calls supplying two, three, and four arguments and label which defaults are used in each case. • Declare defaults in the function declaration only and keep the definition consistent. • Explain why a required parameter cannot follow a parameter with a default value. • Show how to request insurance while retaining standard delivery, acknowledging that C++ has no named arguments. • Validate positive weight and distance and non-negative insurance value. **Minimum validation evidence:** • Test standard uninsured, express uninsured, standard insured, and express insured parcels. • Test zero or negative required inputs and show a clear rejection. • Include one invalid default-argument declaration as a commented diagnostic example and correct it. **Submission for this question:** design note, complete .cpp source, labelled test evidence, and concept reflection.	

15	**Debugging an Ambiguous College Print-Centre API**
	Primary concepts: *Function overloading; default arguments; ambiguity; compiler diagnostics*
	Scenario: A college print centre combines function overloading and default arguments. After an update, ordinary calls such as fee(100) no longer compile, and another declaration places a required parameter after a defaulted one. You have been asked to repair the interface without removing the useful calling options.
	Programming task: Start from the broken interface below, reproduce the compiler diagnostics, and redesign the functions so calls are clear and unambiguous. Your final program must support a basic monochrome job, a custom-rate job, and a colour job with optional binding.
	Required function contract:
	`double fee(int pages);`
	`double fee(int pages, double ratePerPage = 1.0);`
	`double colourFee(int pages = 10, double ratePerPage);`
	Functional and design requirements:
	• Explain precisely why fee(100) matches more than one candidate and why colourFee() violates the trailing-default rule.
	• Propose a corrected overloaded interface that keeps a concise common call and makes custom-rate and colour calls distinguishable.
	• Use at least one default argument in the corrected interface and at least two valid overloads.
	• Implement all corrected functions and print which overload handles each menu choice.
	• Use compiler diagnostics as evidence, but do not leave uncompilable declarations active in the final source file.
	• Discuss why adding explicit casts everywhere would hide a poor interface rather than fully solve the design problem.
	Minimum validation evidence:
	• Demonstrate one-argument, two-argument, and fully specified calls.
	• Check 0 pages, a normal job, and an unusually large job.
	• Compile the repaired version with no ambiguity warnings or errors.
	Submission for this question: design note, complete .cpp source, labelled test evidence, and concept reflection.
		K4, CO1